MICROCONTROLLER-8051

Features & Applications

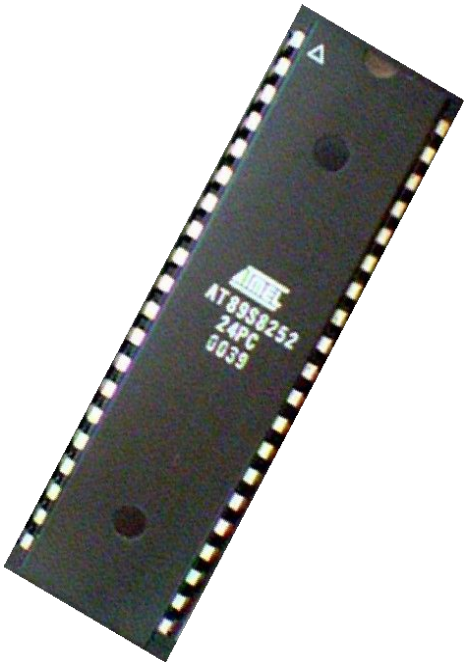
….. Man's glory lies in his knowledge, his upright conduct, his praise-worthy character, his wisdom, and not in his nationality or rank.

—Baha'ullah

(From, the book The 8051 Microcontroller and Embedded systems- Mazidi)

Overview

- **Introduction**
- **Block Diagram and Pin Description of the 8051**
- **Registers**
- **Memory mapping in 8051**
- **Stack in the 8051**
- **I/O Port Programming**
- **Timers**
- **Interrupts & Applications**



Why do we need to learn Microcontrollers ?

- Its not an exaggeration if I say that, today there is no electronic gadget on the earth which is designed without a Microcontroller. Ex: communication devices, digital entertainment, portable devices etc…

Not believable ??? See the next slide

- Personal information products: Cell phone, pager, watch, pocket recorder, calculator
- Laptop components:mouse, keyboard, modem, fax card, sound card, battery charger
- Home appliances:door lock, alarm clock, thermostat, air conditioner, TV remote, VCR, small refrigerator, exercise equipment, washer/dryer, microwave oven
- Industrial equipment: Temperature/pressure controllers, Counters, timers, RPM Controllers
- Toys: video games, cars, dolls, etc.

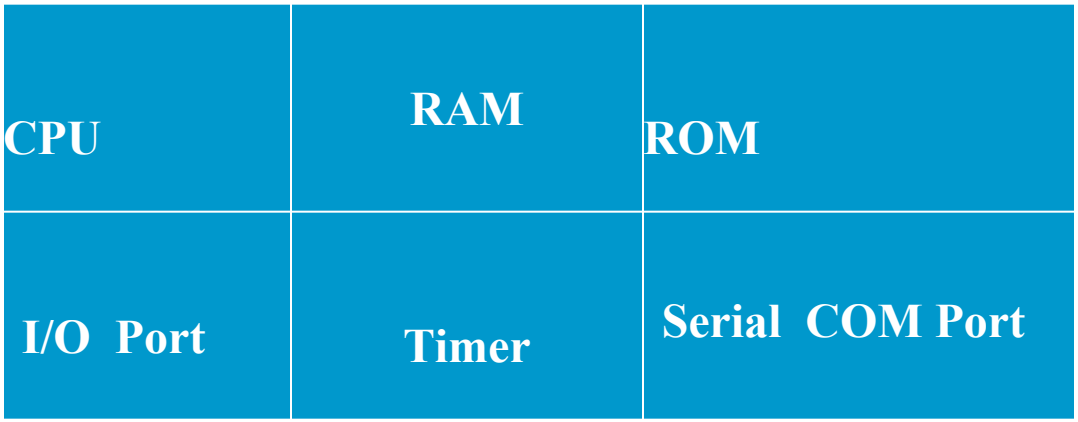
So, A good designer should always know what type of controller he/she is using ,their architecture, advantages , disadvantages , ways to reduce

production costs and product reliability etc….

O.K ????

Then What is a Microcontroller ?

- A smaller computer
- On-chip RAM, ROM, I/O ports...
- Example: Motorola's 6811, Intel's 8051, Zilog's Z8 and PIC 16X

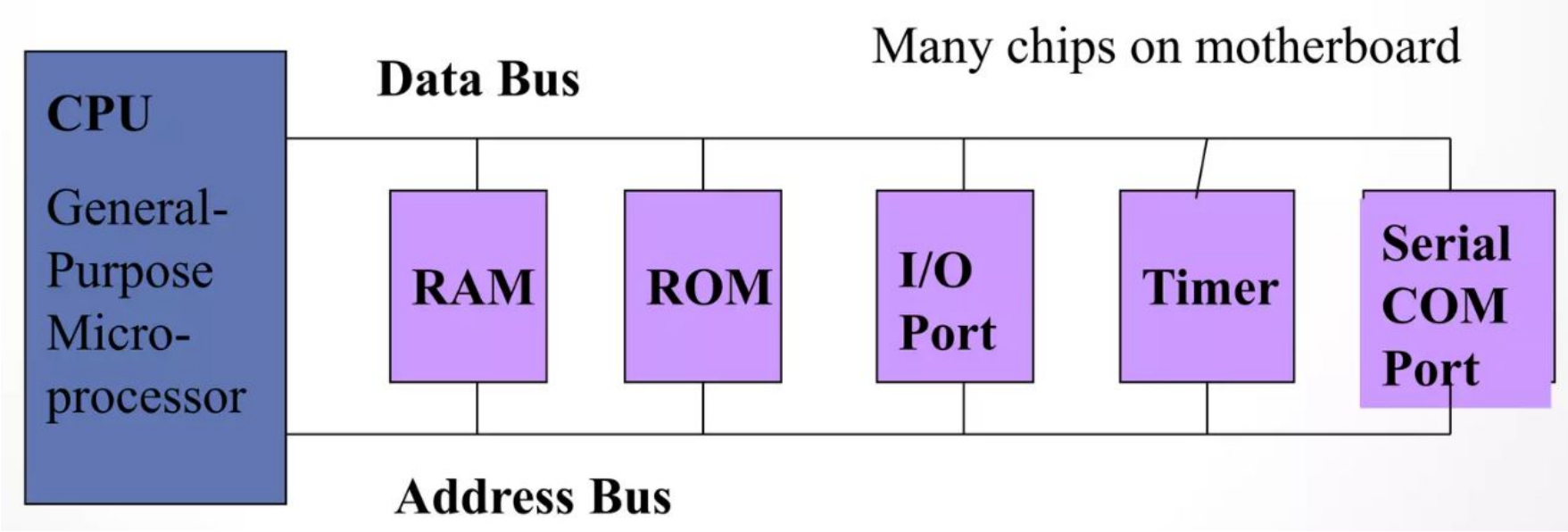


- A single chip Microcontroller

How it is different from a Microprocessor ??

- General-purpose microprocessor

CPU for Computers
No RAM, ROM, I/O on CPU chip itself
Example ： Intel's x86, Motorola's 680x0



Microprocessor vs. Microcontroller

Microprocessor

- CPU is stand-alone, RAM, ROM, I/O, timer are separate
- Designer can decide on the amount of ROM, RAM and I/ O ports.
- Expansive
- Versatility
- General-purpose

Microcontroller

- CPU, RAM, ROM, I/O and timer are all on a single chip
- fix amount of on-chip ROM, RAM, I/O ports
- Highly bit addressable
- for applications in which cost, power and space are critical
- single-purpose

EVOLUTION

Flashback !!!!

- In 1970 and 1971, about the same time Intel was inventing its microprocessor, Gary Boone, an engineer at Texas Instruments was working on a similar idea. This extraordinary breakthrough was given the rather humdrum name of the TMS1802NC. It is named as microcontroller.
- The INTEL bagged the credit of producing the first Microcontroller 8048 with a CPU and 1K bytes of EPROM, 64 Bytes of RAM an 8-Bit Timer and 27 I/O pins in 1976.
- Then followed the most popular controller 8051 in the year 1980 with 4K bytes of ROM,128 Bytes of RAM , a serial port, two 16-bit Timers , and 32 I/O pins.
- The 8051 family has many additions and improvements over the years and remains a most sought after tool for todays circuit designers.

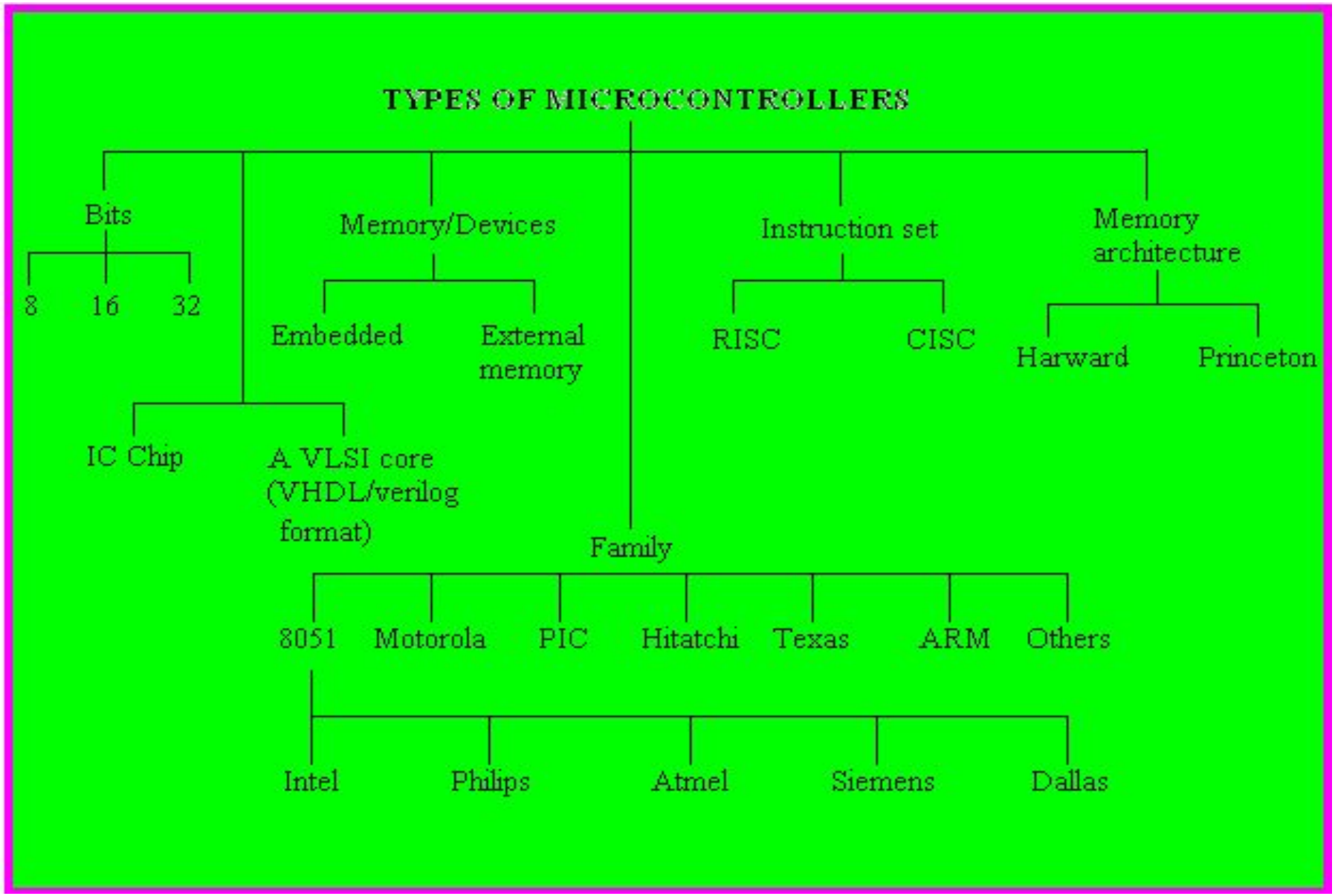
Evolution contd...

- The same INTEL introduced a 16 bit controller 8096 in the year 1982
- Later INTEL introduced 80c196 series of 16-bit microcontrollers for mainly industrial applications
- Microchip, another company has introduced a microcontroller PIC 16C64 an 8-bit in the year 1985.
- 32-bit microcontrollers have been developed by IBM and Motorola-MPC 505 is a 32-bit RISC controller of Motorola
- The 403 GA is a 32 -bit RISC embedded controller

ARM Controllers

- In recent times ARM company (Advanced RISC machines) has developed and introduced 32 bit controllers which are high end application devices, especially communication devices like mobiles , ipods etc..(Refer www.arm.com)

Types of Microcontrollers

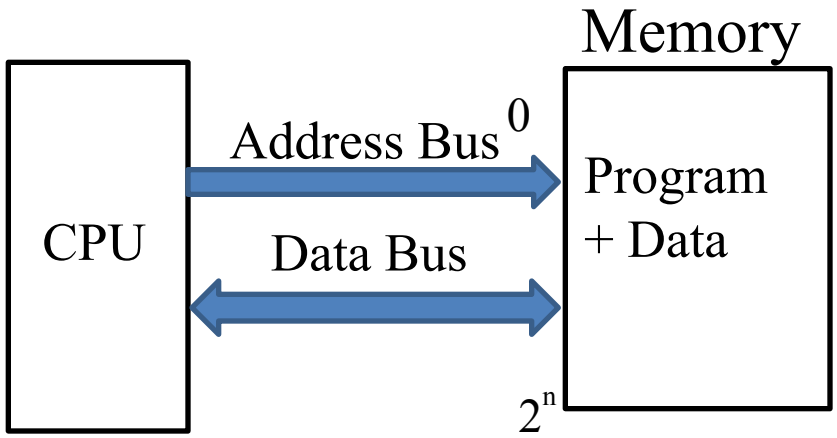


MCS-51 “Family” of Microcontollers

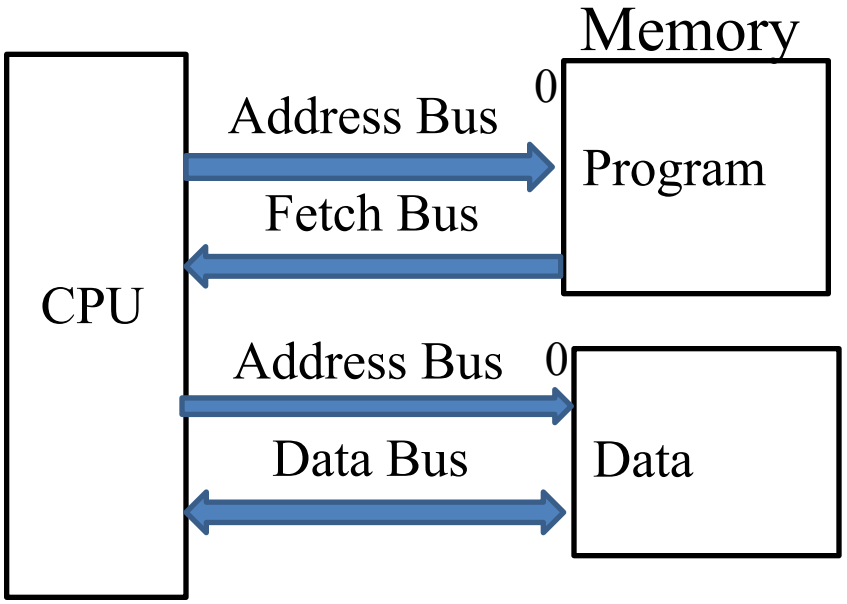
Feature	**8031**	**8051**	**8052**	**8751**
ROM	NO	4kB	8kB	4kB UV Eprom
RAM (Bytes)	128	128	256	128
TIMERS	2	2	3	2
I/O PINS	32	32	32	32
SERIAL PORTS	1	1	1	1
INTERRUPT SOURCES	6	6	8	6

EPROM: Erasable programmable read-only memory

Microcontroller Architectures



Von Neumann Architecture

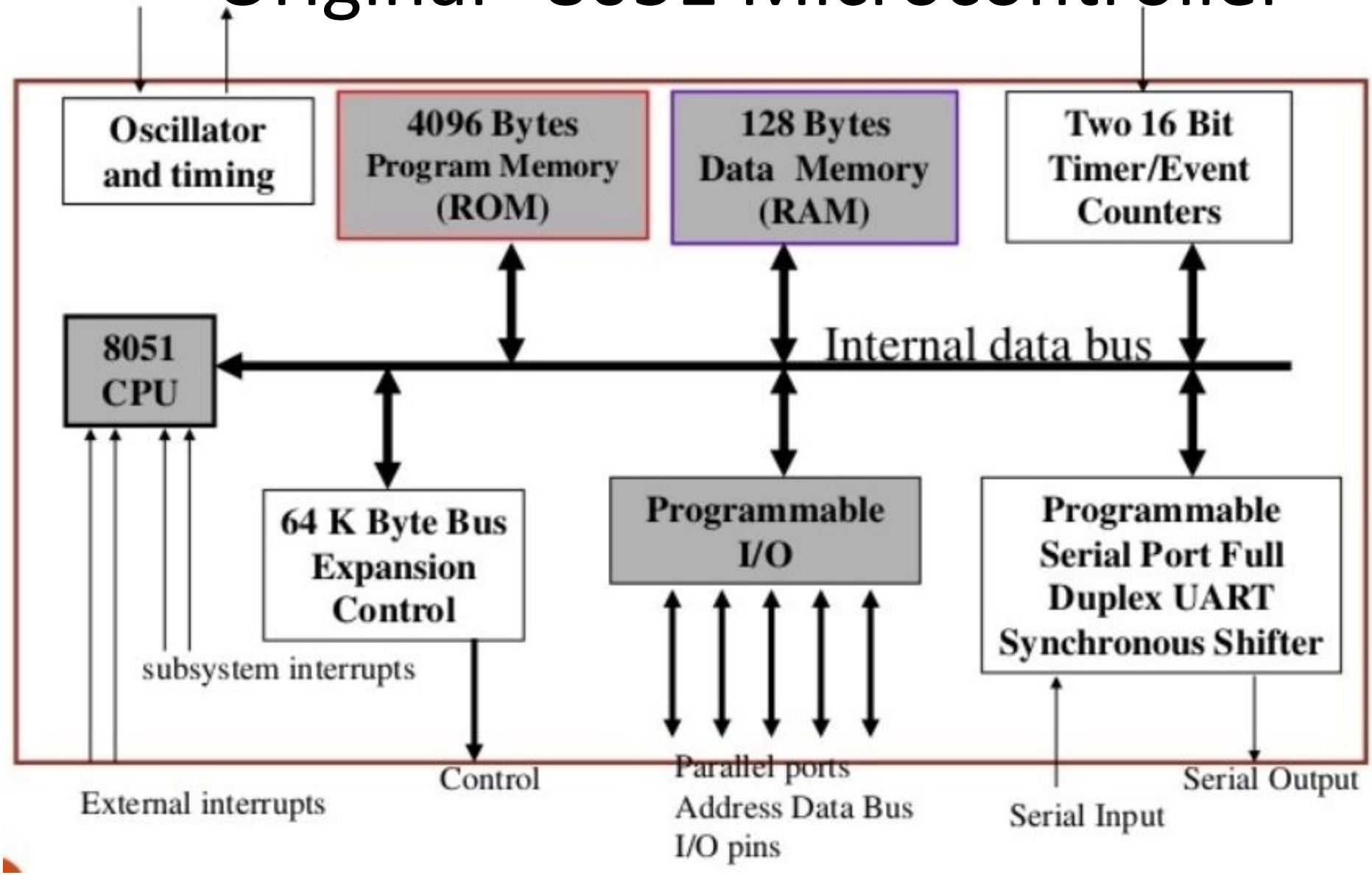


Harvard Architecture

Important Features of8051

- 4K bytes ROM
- 128 bytes RAM
- Four 8-bit I/O ports
- Two 16-bit timers
- Serial interface
- 64K external code memory space
- 64K data memory space

“Original” 8051 Microcontroller



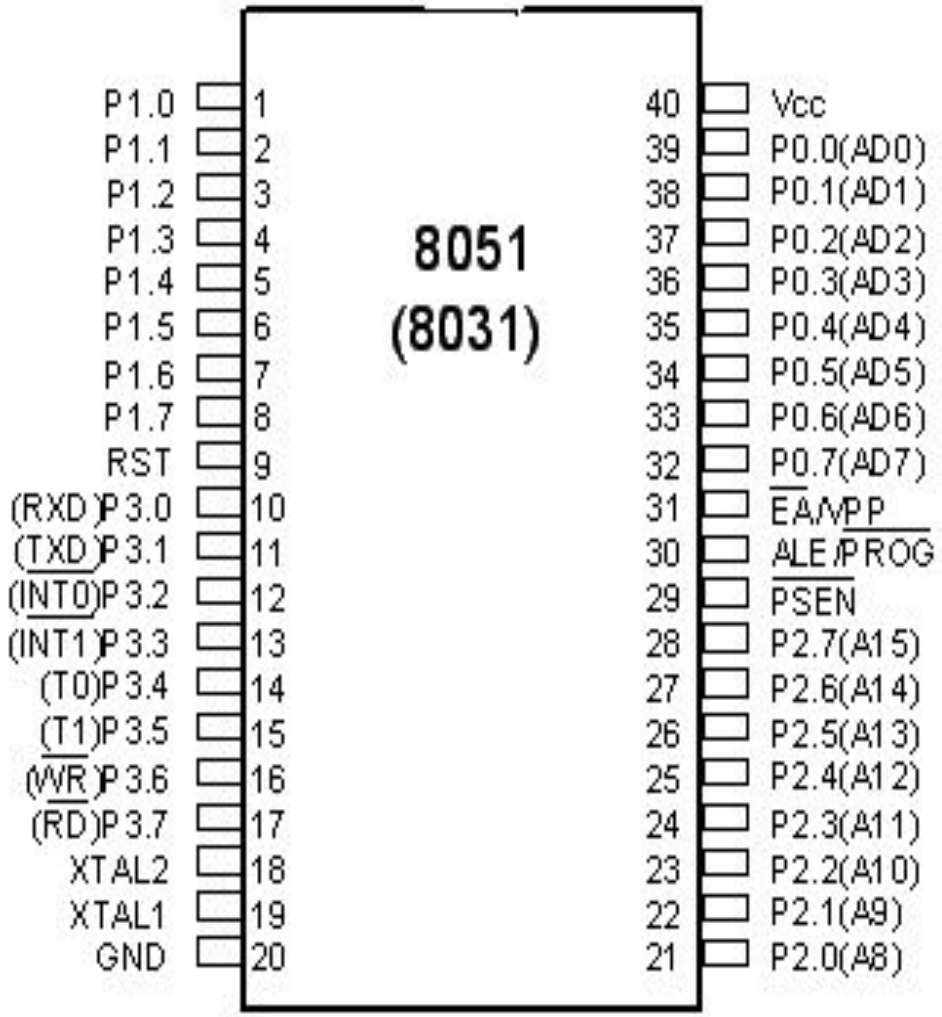
Memory Organization

- 8051 has 2 types of memory
  - Program memory (ROM)
  - Data memory (RAM)
- Few KB of ROM and 128 or 256 bytes of RAM
- 16 address bus capable of addressing 64KB of memory
- External memory can also be used

Pin Description of the 8051

- The 8051 is a 40 pin device, but out of these 40 pins, 32 are used for I/O.

- 24 of these are dual purpose, i.e. they can operate as I/O or a control line or as part of address or date bus.



https://www.tutorialspoint.com/microprocessor/microcontrollers_8051_pin_description.htm

Pin 30 (ALE/PROG): Pin 30 is the Address Latch Enable Pin. Using this Pins, external address can be separated from data (as they are multiplexed by 8051).

During Flash Programming, this pin acts as program pulse input (PROG).

Pin 31 (EA/VPP): Pin 31 is the External Access Enable Pin i.e. allows external program Memory. Code from external program memory can be fetched only if this pin is LOW. Else internal program memory and external program memory is used.

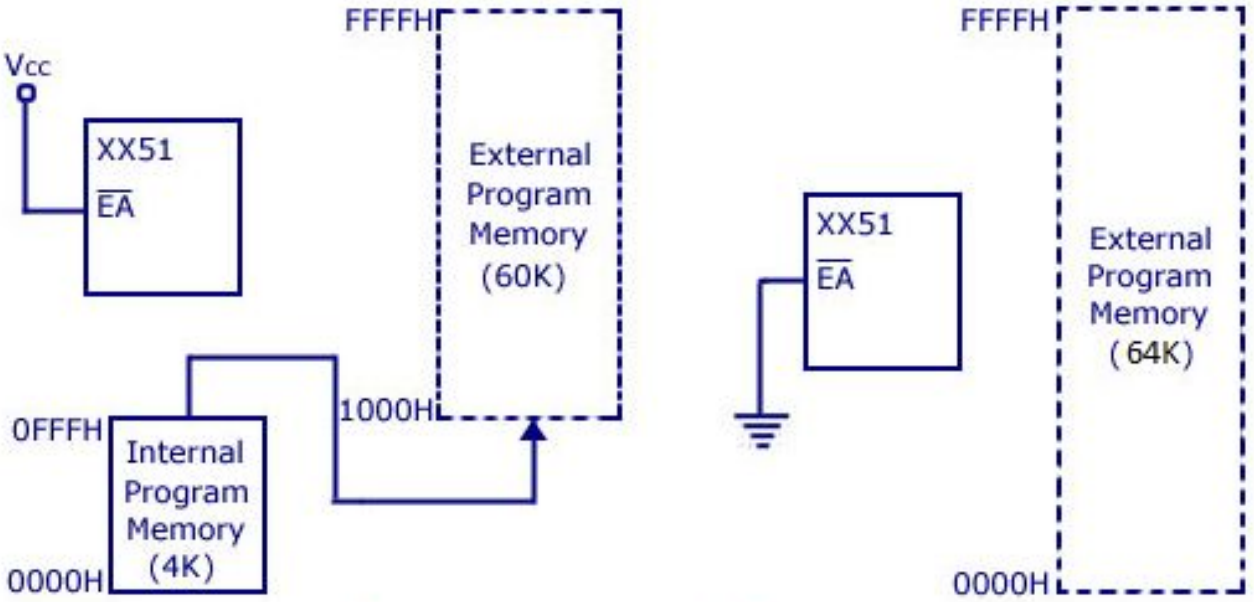
During Flash Programming, this Pin receives 12V Programming Enable Voltage (VPP).

Flash memory is just like EEPROM: Electrically erasable programmable read only memory.

PSEN: This is an output pin. PSEN stands for "program store enable." This pin is used to read external program memory when '0' else external data memory. If we use an external ROM for storing the program, then logic 0 appears on it, which indicates Microcontroller to read data from the memory.

https://www.electronicshub.org/8051-microcontroller-memory-organization/

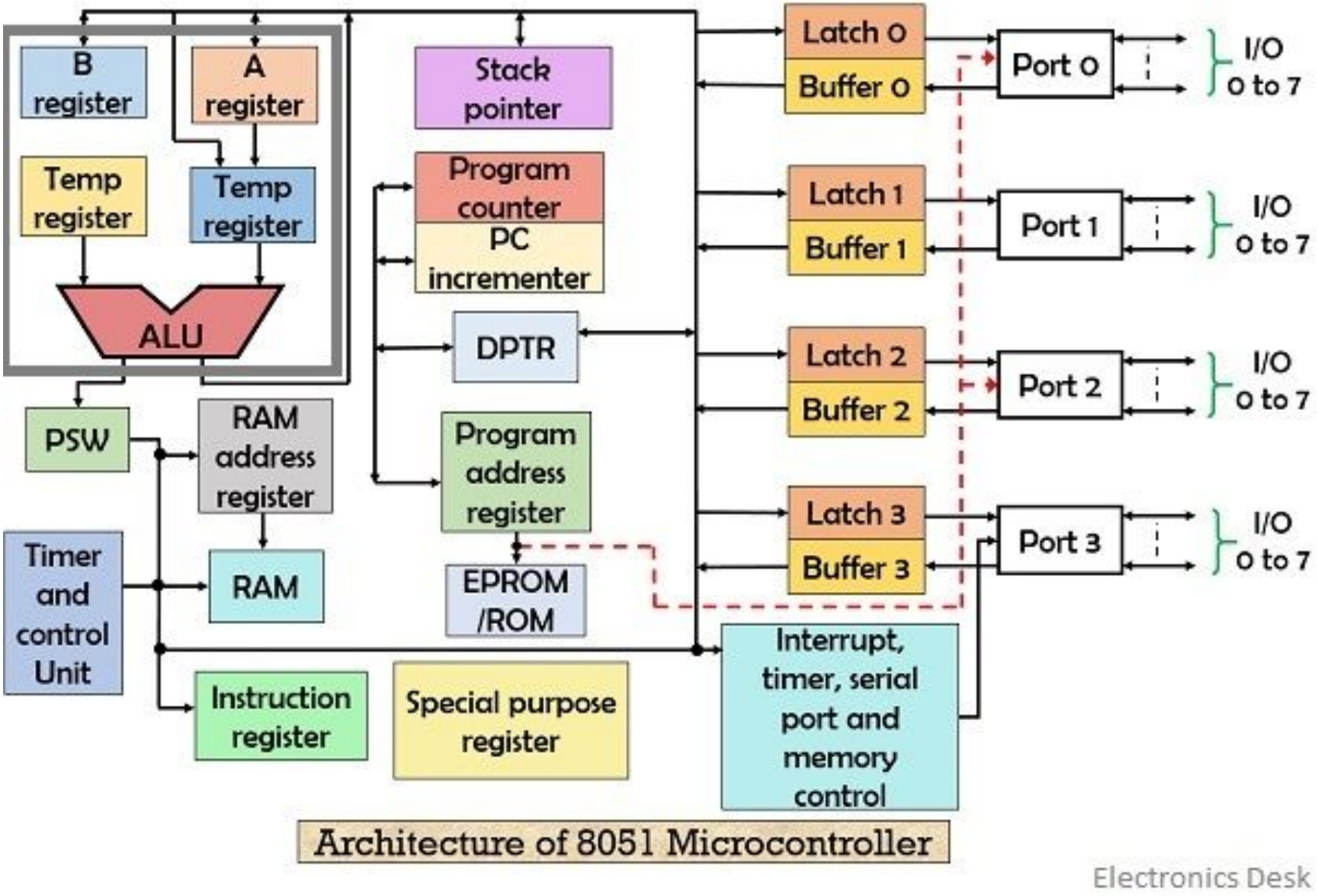
EA :



Program memory organization

PSEN (Pin 29)

- PSEN, **"program store enable"**, is an output pin
- This pin is connected to the OE pin of the external memory.
- For External Code Memory, $\overline{PSEN} = 0$
- For External Data Memory, $\overline{PSEN} = 1$

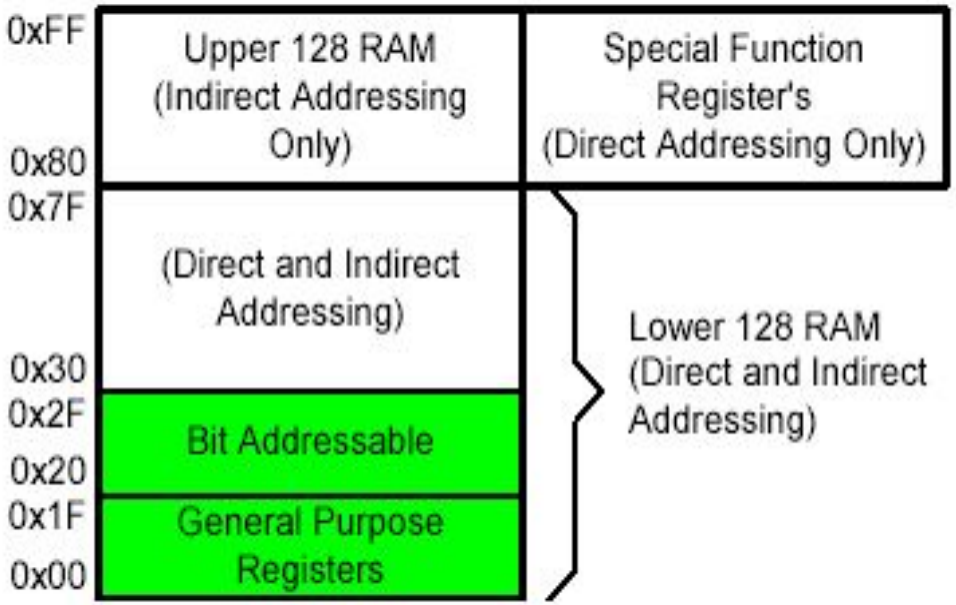


- A (8-bit Accumulator). B (8-bit register for Mul & Div). PSW (8-bit Program Status Word). SP (8-bit Stack Pointer). PC (16-bit Program Counter). DPTR (16-bit Data Pointer)

RAM memory space allocation in the 8051

- Almost all modern variants of 8051 Microcontroller have 256B of RAM. In this 256B, the first 128B i.e. memory addresses from 00H to 7FH is divided into Working Registers (organized as Register Banks), Bit – Addressable Area and General Purpose RAM (also known as Scratchpad area). In the first 128B of RAM (from 00H to 7FH), the first 32B i.e. memory from addresses 00H to 1FH consists of 32 Working Registers that are organized as four banks with 8 Registers in each Bank.

- DATA registers
- CONTROL registers
- Timers
- Serial ports
- Interrupt system

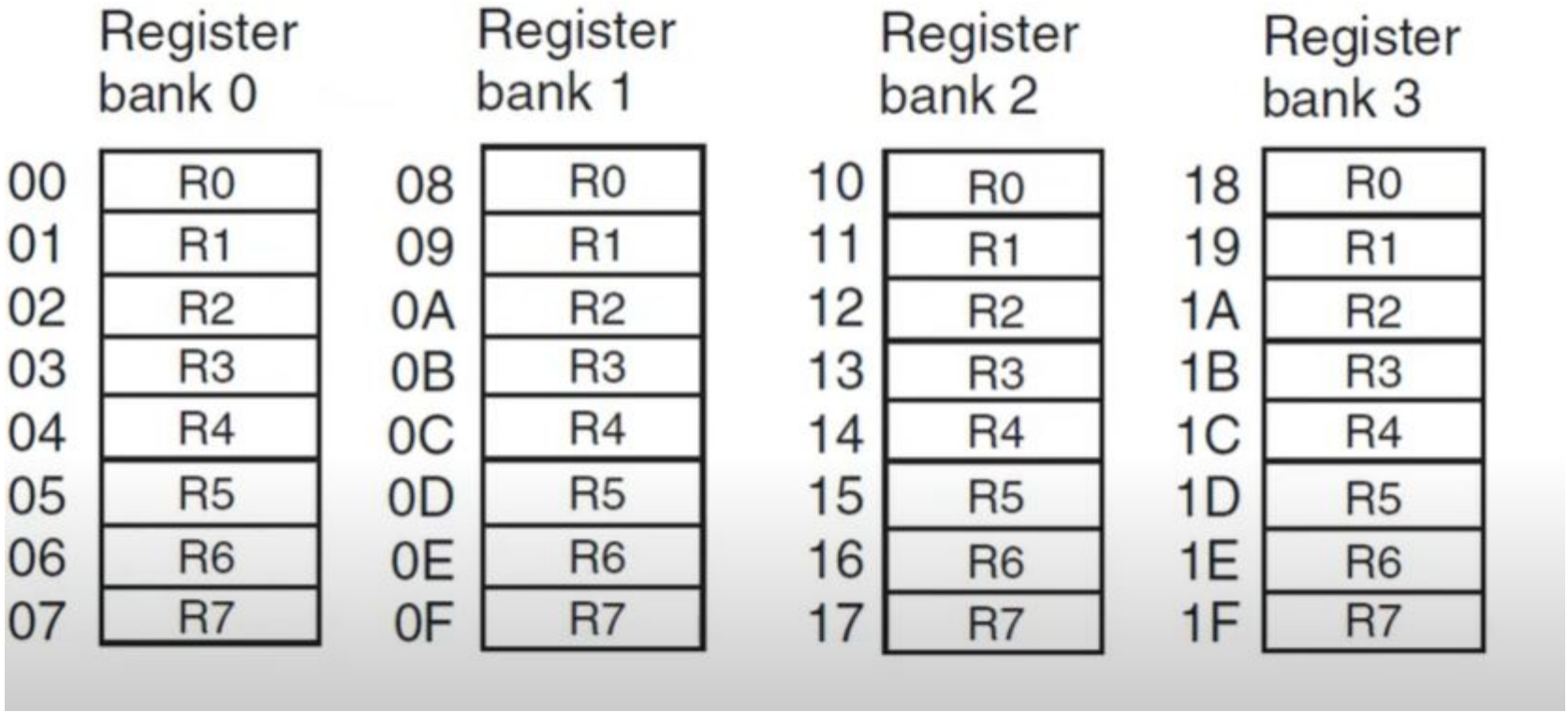


Addresses 80h – FFh

Direct Addressing is used to access SFRs

•Analog to Digital converter
•Digital to Analog converter
etc..

8051 Register bank



Special Function Registers -SFR

- 8051 microcontroller has a dedicated SFR that controls various functionalities and peripheral devices. These registers are essential for programming the microcontroller's behavior.
- Direct Addressing
- 80h to FFh

List of Special Function Registers In 8051 Microcontroller

- Accumulator (A)
- B Register (B)
- Program Status Word (PSW)
- Data Pointer (DPTR)
- Stack Pointer (SP)
- DPL (Data Pointer Low)
- DPH (Data Pointer High)
- Interrupt Enable Register (IE)
- Interrupt Priority Register (IP)
- I/O Port Registers (P0, P1, P2, P3)
- Power Control Register (PCON)
- Serial Control Register (SCON)
- SBUF (Serial Buffer Register)
- Timer/Counter Registers (TMOD, TH0, TL0, TH1, TL1)

Different Types of Special Function Registers (SFRs) in 8051 microcontroller:

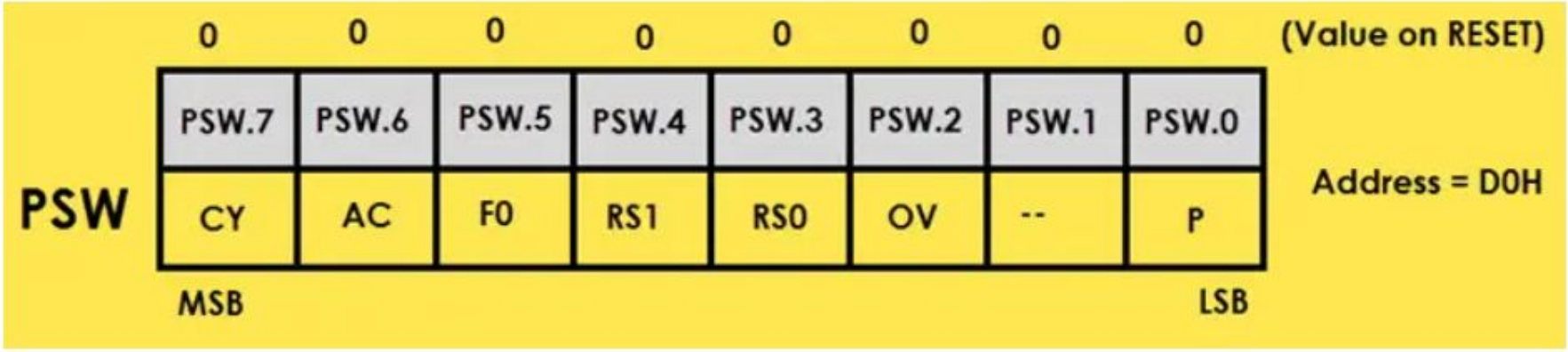
- ***Math or CPU Registers:*** A and B
- ***Status Register:*** PSW (Program Status Word)
- ***Pointer Registers:*** DPTR (Data Pointer – DPL, DPH) and SP (Stack Pointer)
- ***I/O Port Latches:*** P0 (Port 0), P1 (Port 1), P2 (Port 2) and P3 (Port 3)
- ***Peripheral Control Registers:*** PCON, SCON, TCON, TMOD, IE and IP
- ***Peripheral Data Registers:*** TL0, TH0, TL1, TH1 and SBUF

Name of the Register	*Function*	*Internal RAM Address (HEX)*
ACC	Accumulator	E0H
B	B Register (for Arithmetic)	F0H
DPH	Addressing External Memory	83H
DPL	Addressing External Memory	82H
IE	Interrupt Enable Control	A8H
IP	Interrupt Priority	B8H
P0	PORT 0 Latch	80H
P1	PORT 1 Latch	90H
P2	PORT 2 Latch	A0H
P3	PORT 3 Latch	B0H
PCON	Power Control	87H
PSW	Program Status Word	D0H
SCON	Serial Port Control	98H
SBUF	Serial Port Data Buffer	99H
SP	Stack Pointer	81H
TMOD	Timer / Counter Mode Control	89H
TCON	Timer / Counter Control	88H
TL0	Timer 0 LOW Byte	8AH
TH0	Timer 0 HIGH Byte	8CH
TL1	Timer 1 LOW Byte	8BH
TH1	Timer 1 HIGH Byte	8DH

ELECTRONICS HUB

Program Status word

- The PSW or Program Status Word Register is also called as Flag Register and is one of the important SFRs. The PSW Register consists of Flag Bits, which help the programmer in checking the condition of the result and also make decisions.



BIT	SYMBOL	FLAG NAME			DESCRIPTION
7	C or CY	Carry			Used in Arithmetic, Logic & Boolean Operations
6	AC	Auxiliary Carry			Used in BCD Arithmetic
5	F0	Flag 0			General Purpose User Flag
4	RS1	Register Bank Selection Bit 1			
3	RS0	Register Bank Selection Bit 1			
		RS1	RS0	Bank	
		0	0	Bank 0	
		0	1	Bank 1	
		1	0	Bank 2	
		1	1	Bank 3	
2	OV	Overflow			Used in Arithmetic Operations
1	--	Reserved			May be used as a General Purpose Flag
0	P	Parity			Set to 1 if A has odd # of 1's; otherwise Reset

Accumulator

- The Register A is located at the address E0H in the SFR memory space. The Accumulator is used to hold the data for almost all the ALU Operations



B Register

- The B Register is used along with the ACC in Multiplication and Division operations.
- Register B is located at the address F0H of the SFR Address Space.



PORTS OF 8051

- 8051 has 4 Ports. Port 0, Port1, Port2 , Port3
- Port 0 is a dual purpose port, it is located from pin 32 to pin 39 (8 pins). To use this port as both input/output ports each pin must be connected externally to a 10 k ohm pull-up resistor. This is because Port 0 is an open drain.

Simple ex:

```
      MOV A, #22
BACK  MOV P0 ,A
      ACALL DELAY
      CPL A
      SJMP BACK
```

Ports….

- Port 1 is a dedicated I/O port from pin 1 to pin 8.Upon reset it is configured as out port. It is generally used for interfacing to external device thus if you need to connect to switches or LEDs, you could make use of these 8 pins,but it doesn't need any pull-up resistors as it is having internally

- Like port 0, port 2 is a dual-purpose port.(Pins 21 through 28) It can be used for general I/O or as the high byte of the address bus for designs with external code memory.Like P1 ,Port2 also doesn't require any pull-up resistors

Ports….

- Port 3 is also dual purpose but designers generally avoid using this port unnecessarily for I/O because the pins have alternate functions which are related to special features of the 8051. Indiscriminate use of these pins may interfere with the normal operation of the 8051.

- However, for a programmer, it is the same to program P0, P1, P2 and P3.

- All the ports upon RESET are configured as output. To use any of the ports as an input port,it

Addressing Modes

- Eight modes of addressing are available with the C8051F020
- The different addressing modes determine how the operand byte is selected

Addressing Modes	Instruction	
Register	MOV	A, B
Direct	MOV	30H,A
Indirect	ADD	A,@R0
Immediate Constant	ADD	A,#80H
Relative*	SJMP	AHEAD
Absolute*	AJMP	BACK
Long*	LJMP	FAR_AHEAD
Indexed	MOVC	A,@A+PC

* Related to program branching instructions

Register Addressing

- The register addressing instruction involves information transfer between registers

- *Example*:

```
MOV     R0, A
```

- The instruction transfers the accumulator content into the R0 register. The register bank (Bank 0, 1, 2 or 3) must be specified prior to this instruction.

Direct Addressing

- This mode allows you to specify the operand by giving its actual memory address (typically specified in hexadecimal format) or by giving its abbreviated name (e.g. P3)

  Note: Abbreviated SFR names are defined in the “C8051F020.inc” header file

- *Example*:

```
MOV   A, P3     ;Transfer the contents
                of
                ;Port 3 to the
MOV   A, 020H   accumulator
                ;Transfer the contents of location 20H to the
                accumulator
                of RAM
```

Indirect Addressing

- This mode uses a pointer to hold the effective address of the operand
- Only registers R0, R1 and DPTR can be used as the pointer registers
- The R0 and R1 registers can hold an 8-bit address, whereas DPTR can hold a 16-bit address
- ***Examples***:

```
                    ;Store the content of
                    ;accumulator into the memory
MOV @R0,A         ;location pointed to by
     ;register R0. R0 could have an
;8-bit address, such as 60H.
```

```
MOVX    A,@DPTR   ;Transfer the contents from
                  ;the memory location
                  ;pointed to by DPTR into the
                  ;accumulator. DPTR could
                  have a
     ;16-bit address, such as 1234H.
```

Immediate Constant Addressing

- This mode of addressing uses either an 8- or 16-bit constant value as the source operand
- This constant is specified in the instruction, rather than in a register or a memory location
- The destination register should hold the same data size which is specified by the source operand

- *Examples*:

```
ADD A,#030H        ;Add 8-bit value of 30H to
                   ;the accumulator register
                   ;(which is an 8-bit register).

MOV                ;Move 16-bit data constant
DPTR,#0FE00H       ;FE00H into the 16-bit Data
                   ;Pointer Register.
```

Relative Addressing

- This mode of addressing is used with some type of jump instructions, like SJMP (short jump) and conditional jumps like JNZ
- These instructions transfer control from one part of a program to another
- The destination address must be within -128 and +127 bytes from the current instruction address because an 8-bit offset is used ($2^8 = 256$)
- *Example*:

```
GoBack:   DEC     A       ;Decrement A
          JNZ     GoBac   ;If A is not zero,
                  k       loop back
```

Absolute Addressing

- Two instructions associated with this mode of addressing are ACALL and AJMP instructions
- These are 2-byte instructions where the 11-bit absolute address is specified as the operand
- The upper 5 bits of the 16-bit PC address are not modified. The lower 11 bits are loaded from this instruction. So, the branch address must be within the current 2K byte page of program memory (2^{11} = 2048)
- *Example*:

```
                                    ;PORT_INIT
                                    should be
        ACALL  PORT_INIT
                                    ;located within 2k
                                    bytes.

PORT_INIT:      P0, #0FH    ;PORT_INIT
MOV                         subroutine
```

Long Addressing

- This mode of addressing is used with the LCALL and LJMP instructions
- It is a 3-byte instruction and the last 2 bytes specify a 16-bit destination location where the program branches
- It allows use of the full 64 K code space
- The program will always branch to the same location no matter where the program was previously
- *Example*:

```
        LCALL TIMER_INIT          ;TIMER_INIT address (16-bits
                                  ;long) is specified as the
                                  ;operand; In C, this
                                  ;TIMER_INIT will be a
                                  subroutine

TIMER_INIT: ORL
TMOD,#01H

                                  ;function call:
                                  Timer_Init().
```

Indexed Addressing

- The Indexed addressing is useful when there is a need to retrieve data from a look-up table
- A 16-bit register (data pointer) holds the base address and the accumulator holds an 8-bit displacement or index value
- The sum of these two registers forms the effective address for a JMP or MOVC instruction
- *Example*:

```
MOV     A,#08H          ;Offset from table start
MOV     DPTR,#01F0H     ;Table start address
MOVC    A,@A+DPTR       ;Gets target value from the
        table start address + offset and puts it
;in A.
```

- After the execution of the above instructions, the program will branch to address 1F08H (1F00H+08H) and transfer into the accumulator the data byte retrieved from that location (from the look-up table)

Keil software introduction:

https://www.youtube.com/watch?v=mhHJV21CDjs.